No of additional sheets

LOURDES MATHA COLLEGE OF SCIENCE & TECHNOLOGY

LOURDES HILLS, KUTTICHAL P. O., THIRUVANANTHAPURAMM -,695 574

ANSWER BOOK

Name Karya Chandran G.R. Class No. 23 Branch MCA

Subject Deep Learning Semester 3

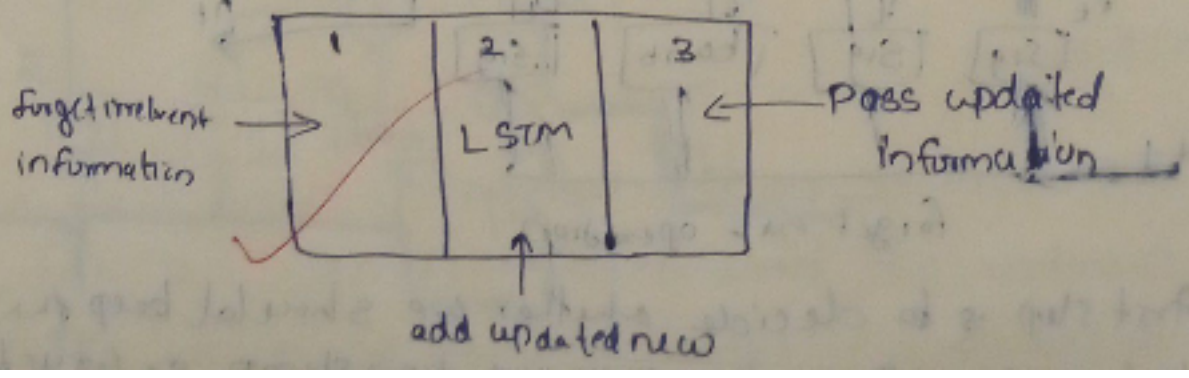
Mark Tabulation

		Q.1	Q.2	Q.3	Q.4	Q.5	Q.6	Q.7	Q.8	Q.9	Q.10	Q.11	Q.12	Q.13	Q.14	Q.15
Mark Divisions		3	2	3	3	3	3		6	6						
Total																
Grand Total														29		

8. Long short Term Memory (LSTM)

LSTM network is an advanced RNN, a sequential network that allows information to persist. It is capable of handling the vanishing gradient problem faced by RNN.

LSTM consist of 3 parts forgetgat, Inputgat, outputgat



first part chooses whether [illegible] coming from the previous timestamp is to be remembered or is irrelevant and can be forgotten. In the second part, the cell tries to learn new information from the input to this cell. At last cell passes the updated information from the current timestamp to the next timestamp.

Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

t = timestamp

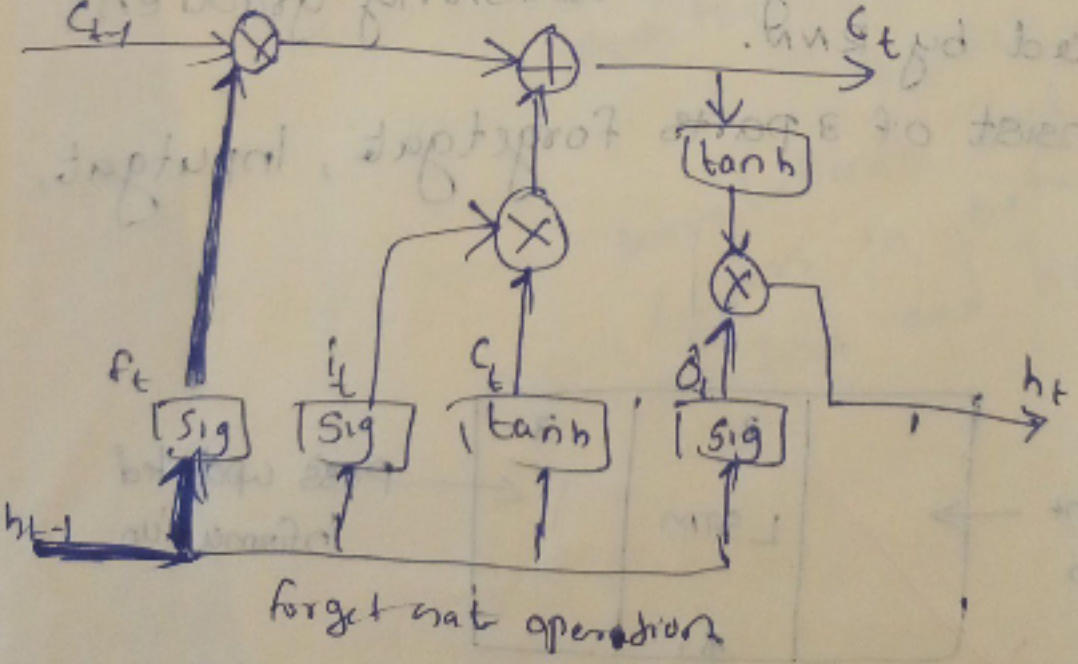
f_t = forget gate at t

x_t = input

h_{t-1} = previous hidden state

W_f = weight matrix

b_f = connection bias at t



First step is to decide whether we should keep the information from the previous timestamp or forget it

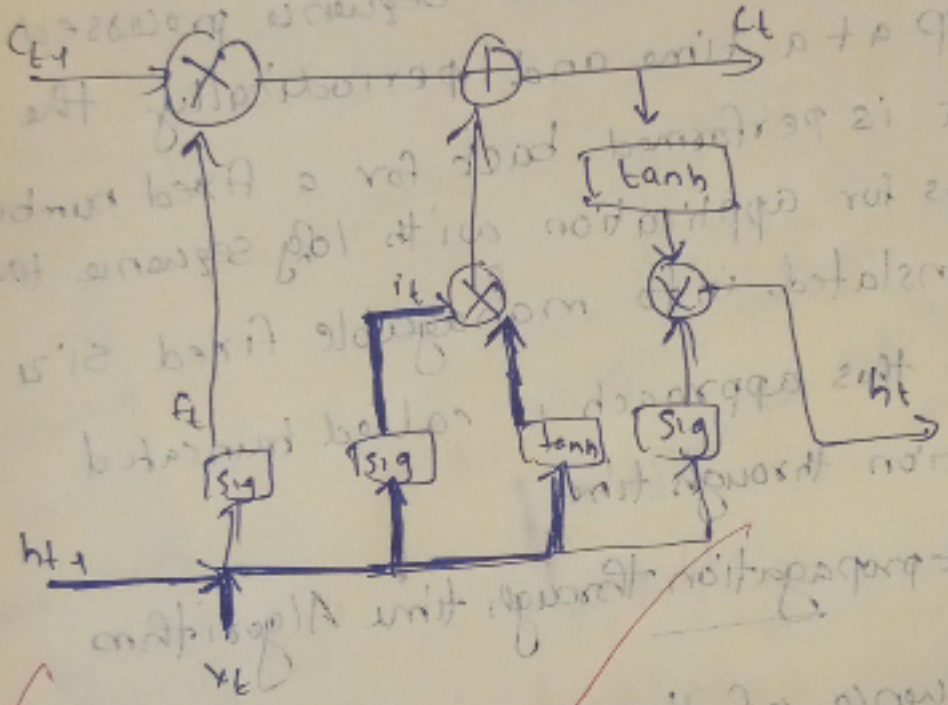
$C_{t-1} * f_t = 0$ if $f_t = 0$ (forget everything)

$C_{t-1} * f_t = C_{t-1}$ if $f_t = 1$ (forget nothing)

input gate

$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$

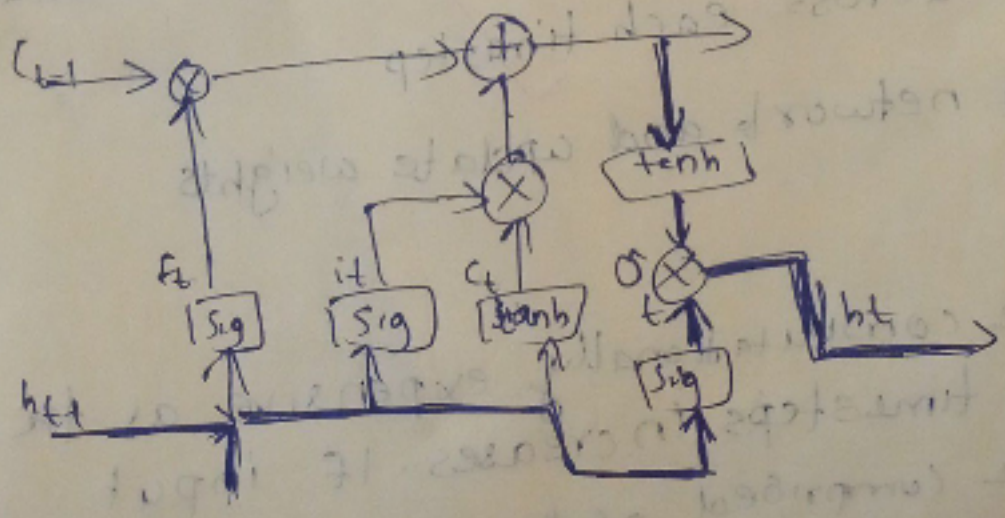
$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$



output gate:

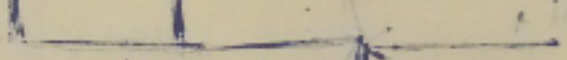
$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$

$h_t = o_t * \tanh(C_t)$



9. Back propagation Through Time is the application of the back propagation training algorithm to ~~Refer~~ recurrent neural network applied to sequence def like a time series.

→ Truncated Backpropagation Through time is a modified version of the BPTT training algorithm for recurrent neural network sequence processed one time step at a time and periodically the BPTT update is performed back for a fixed number of time steps for application with long sequence the input is translated into manageable fixed size segments and this approach is called truncated backpropagation through time.



Truncated backpropagation through time Algorithm

1) present a sequence of timesteps of input and output pairs to the network
2) unroll the network then calculate ~~the~~ and accumulate the errors across each timestep
3) Roll up the network and update weights
4) Repeat

BPTT can be computationally expensive as the number of timesteps increases. If input sequences are comprised of th

then this will be the number of derivative required for a single update weight update. This can cause weights to vanish or explode and make slow learning.

Training algorithm

1) Present a training input pattern and propagate it through the network get an output.

2) Compare the predicted output to the expected outputs and calculate error.

3) Calculate the derivatives of the error with respect to the network weights.

4) Adjust the weights to minimize the error

5) Repeat

6

if we have a long sequence instead of doing back propagation or forward pass till the end of the sequence do the error calculations and back propogation on chunck. Once chunk 1 is computed, feed the value to the next chucle as input. Hence the hidden values of the previous chunks are fed as the output to the next one. Move on the next chunk and so on. This truncated backpropagation through time.

1. Maxpooling is used to reduce the image size by mapping the size of a given window into a single result by taking the maximum value of the elements in the window

2	0	1	1
0	1	0	0
0	0	1	0
0	3	0	0

maxpooling with a 2×2 window and Stride 2

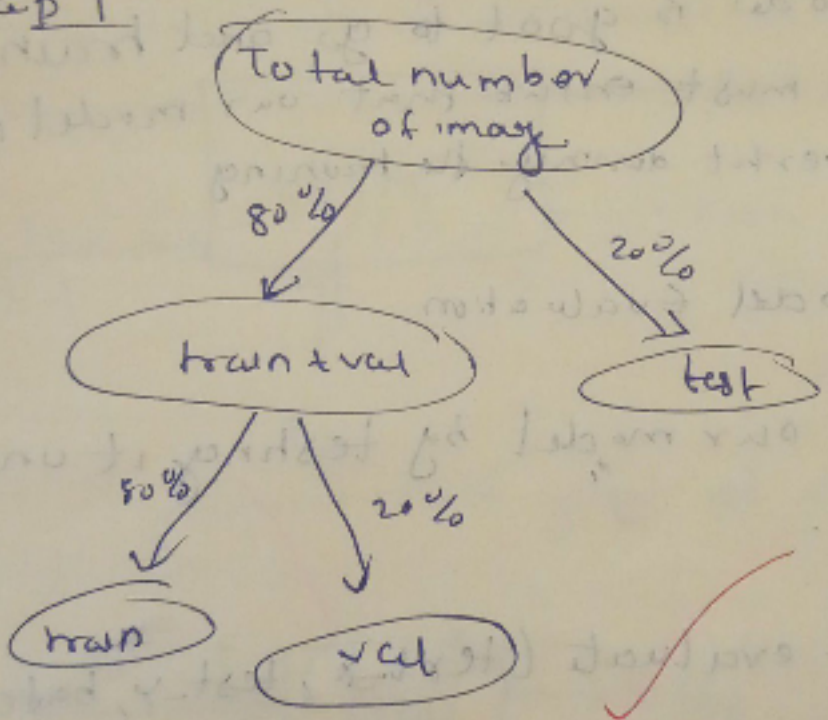
2	1
3	1

3 Pooling is a separate step from convolution. Pooling is used to reduce the size of width and height. the depth is determined by the number of channels. it picks the maximum value in a certain size of the window. it usually applied spatially to reduce the x, y dimensions of an image.

4 steps to develop image build classifier

step 1 collecting your dataset
step 2 preprocessing of the imag
step 3 Model training
step 4 Model evaluation

step 1



step 2

we will be training a VGG 16 model on our custom training dataset to classify among the 3 categories, rock, paper and scissors. pre trained CNN mode input a color image dimension 224×224 of 3 hand gestures

Step 3: Model training

This step includes model building, model compilation and fitting the model.

- model building is using VGG 19 pre trained mode to classify rock paper and scissors
- Compiling is labels of none of the dataset are one hot encoded we should use sparse categorical CROSS entropy as our loss function
- fitting model is good to go and train our mode we must ensure that our model does not get overfit during the training

Step 4: Model Evaluation

evaluate our model by testing it on the test dataset.

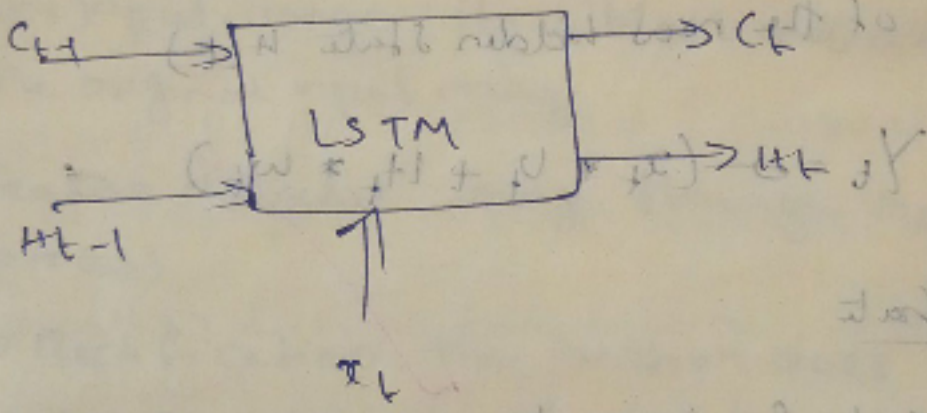
code =>

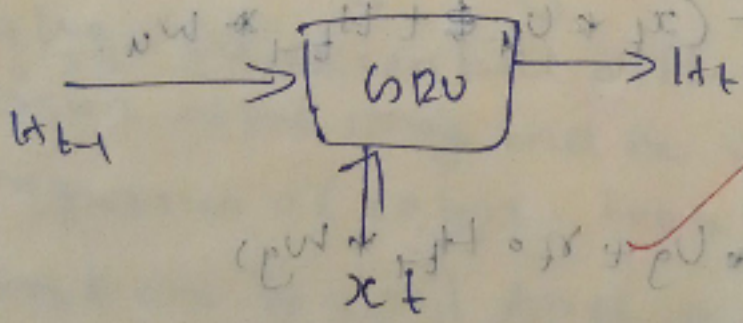
```
model.evaluate(test_x, test_y, batch_size=32)
```

to call our deep learning model good and efficient it is not only accurate enough to locate in step 4.

6. GRU

Gated Recurrent Units is similar as LSTM GRU use gates to control the flow of information. They are relatively new as compared to LSTM. This is the reason they offer some improvement over LSTM have simple architecture.





GRUs cell which more or less similar to the LSTM cell or RNN cell. At each timestamp t_1, it takes as input x_t and the hidden state H_{t-1} from the previous timestamp $t-1$ & later it computes a new hidden state H_t which again passed to the next time stamp.

Reset Gate

Reset Gate is responsible for the short term memory of the new hidden state $H(t)$

$$Y_t = \sigma(x_t * U_t + H_t * W_t)$$

Update Gate

Update Gate for long term memory

$$U_t = \sigma(x_t * U_u + H_{t-1} * W_u)$$

GRU works

$$\hat{H}_t = \tanh(x_t * U_g + r_t \circ H_{t-1} * W_g)$$

Hidden State

$$H_t = U_t \circ H_{t-1} + (1 - U_t) \circ \hat{H}_t$$

2) Rectified linear activation function or ReLU for short is a piecewise linear function that will output the input directly if it is positive, it will output zero. It has become the default activation function for many types of neural networks because a model that uses it is easier to train and often achieves better performance.

→ The input image : it is black and white image is the original input image.

→ Feature detector : image through the convolution process

→ Rectification, the function does to an image like this is remove all the black elements from image keeping only those carrying a positive value. The essential difference blw non rectified version of this image and the rectified one is the progression of colors. Look closely at the first one you will find parts where a white streak is followed by a grey one and then a black one.

2

After we rectify the image you will find the colors changing more abruptly. The gradual change is no longer there. That indicates that the linearity has been disposed.

5) VGG 16 Architecture

VGG stands for visual Geometry Group. The VGG architecture consists of blocks, where each block is composed of 2D convolution and max pooling layer. VGG Net comes in two flavour, VGG16 and 19 the number of layer in each of her respectively.

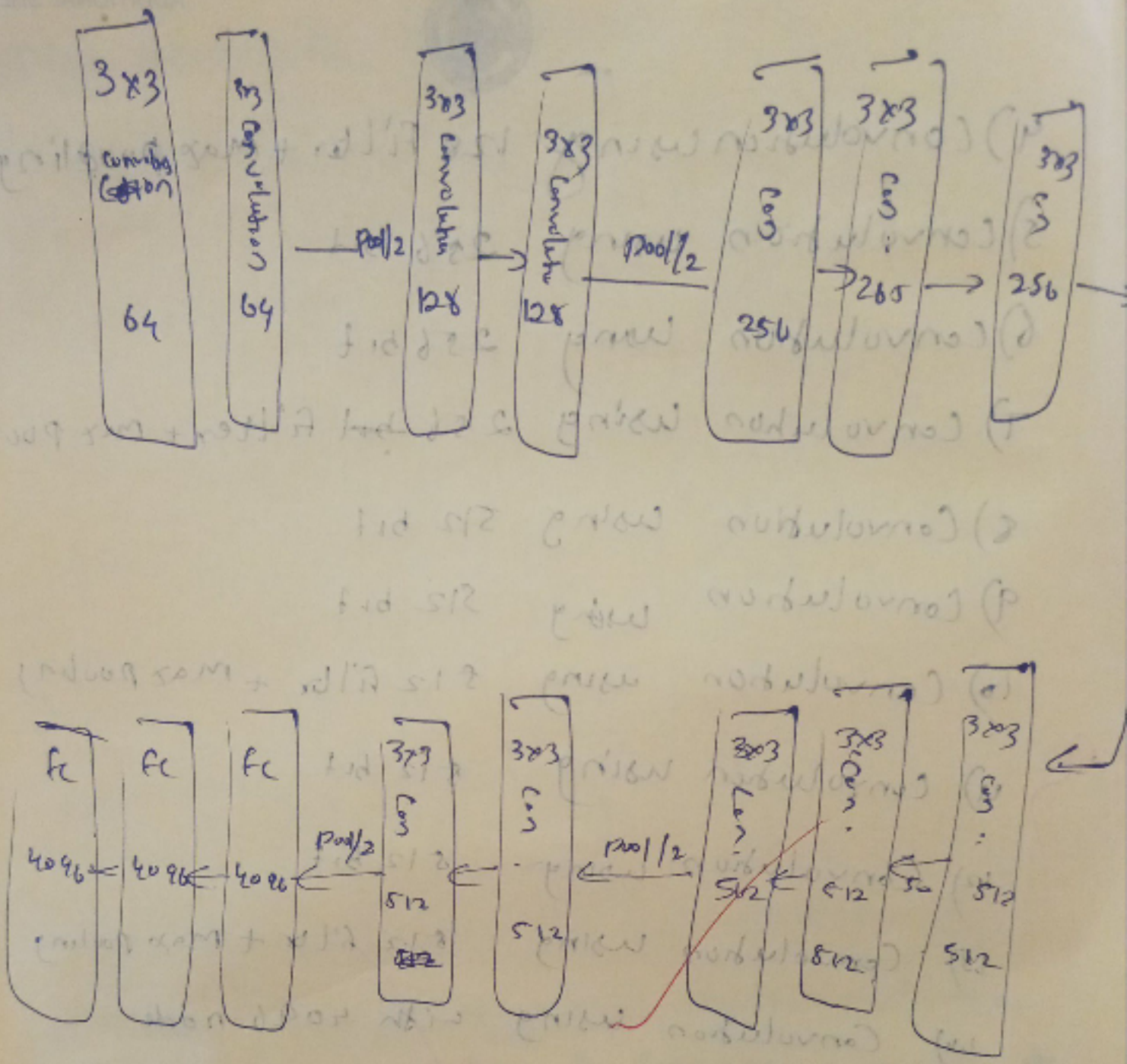
VGG 16 is a convolutional neural network model proposed by K. Simonyan and A. Zisserman. This model of VGG 16 consist of 16 layer they are listed below

16 Layer

1) Convolution using 64 bits
2) Convolution using 64 filter + max pooling
3) Convolution using 128 bits

ADDITIONAL SHEET

4) Convolution using 128 Filter + Max Pooling
5) Convolution using 256 bit
6) Convolution using 256 bit
7) Convolution using 256 ~~bit~~ filter + Max Pooling
8) Convolution using 512 bit
9) Convolution using 512 bit
10) Convolution using 512 filter + Max Pooling
11) Convolution using 512 bit
12) Convolution using 512 bit
13) Convolution using 512 filter + Max Pooling
14) Convolution using with 4096 node
15) Convolution using with 4096 nodes
16) ~~Convolution using~~ output layer with softmax activation with 1000 node


3x3
convolution
64
3x3 convolution 64
pool/2
3x3 convolution 128
3x3 convolution 128
pool/2
3x3 Con 256
3x3 Con 256
3x3 Con 256
fc 4096
fc 4096
fc 4096
pool/2
3x3 Con 512
3x3 Con 512
pool/2
3x3 Con 512
3x3 Con 512
3x3 Con 512
size 7
size 14
size 28

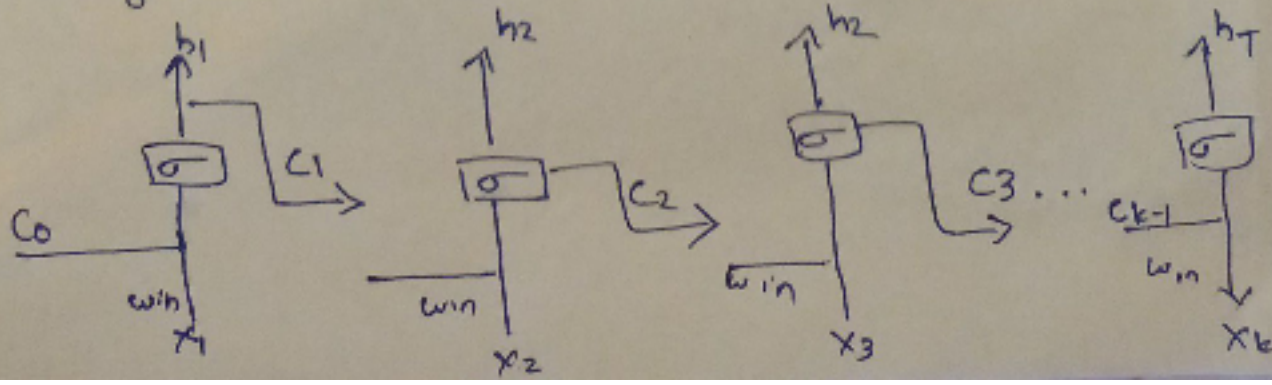
3) problem of vanishing gradients with LSTMs

Two main question arise in recurrent Neural N/w and LSTM network.

1) Why do RNNs suffer from vanishing & Exploding gradients?

2) How do LSTMs keep the gradients from vanishing or explode?

RNN enable modelling time dependant & sequential data tasks such as stock market production, machine translation. RNN suffer from the problem of vanishing gradient which hampers learning of long data sequences. The gradient carry at information use in the RNN parameter update and when the gradient becomes smaller the parameter updates become insignificant which means no real learning is done.



The network has an input sequence of vectors $[x(1), x(2), \cdots x(k)]$ at timestep t the network has an input vector $x(t)$. past information and learned knowledge is encoded in the network state vectors $[c(1), c(2) \cdots c(k-1)]$ at timestep t the network has an input state vector $c(t-1)$. The input vector $x(t)$ and the state vector $c(t-1)$ are concatenated to compose the complex input vector at timestep t.

